

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_fc91ae6e-b62\agent-  
a1c22e9e76a398c0a.jsonl`  
- SHA-256: `3b848eb3c90b3f3fbd1d1973c33e1c0ca4f1d72dfd8e592df08d628c8ee8a8b6`  
- Source modified: `2026-06-14T06:51:33+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_fc91ae6e-b62`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-14T06:50:14.734Z)

In the repo (cwd), extract the ENGINEERING + REPRODUCIBILITY story. Read backend/utils/run_telemetry.py (the telemetry black box just merged in #134), backend/eval/fusion_golden.py (the replayable per-video feature substrate), backend/eval/experiment.py (Variant/compare/record_experiment leaderboard), and note the CI gates (ruff lint, mypy, syntax+import smoke, full offline-mocked suite). Return: how the replayable feature substrate ("persist once, re-score forever") + the experiment leaderboard make results reproducible; the rotating ~10KB telemetry runlog (fps + GPU temp/clocks + CPU/RAM) for forensic post-mortems on long GPU runs; the default-OFF + CI-gated discipline. Brief.

2. assistant (2026-06-14T06:50:17.242Z)

I'll read the four files in parallel to extract the engineering and reproducibility story.

3. user (2026-06-14T06:50:17.726Z)

```
1      """Structured run telemetry ? a compact, rotating JSONL "black box" for long/flaky runs.  
2  
3      A long GPU/CV run (the fusion-golden extraction, a WSL/WASB pass, an ffmpeg stitch) can  
die with  
4      **no Python traceback** ? the OS kills it (a console-control event, an OOM, a thermal  
event). When  
5      that happens the only forensic evidence is whatever was last flushed to disk. This  
module keeps a  
6      tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem  
is a  
7      bullseye instead of a guess: "it was at 26 fps, GPU 85 °C throttled to 1125/2100 MHz,  
4.8 GB VRAM  
8      free, CPU 60 %, RAM 21/32 GB ? then vanished."  
9  
10     It is the **observability** companion to the `ExecutionPolicy` *control* layer  
11     (`docs/EXECUTION_POLICY.md`): the policy decides wait / abort / resume; this records the  
signals  
12     that explain *why* a run slowed or died, and lets us compare execution velocity across  
quality  
13     iterations.  
14  
15     Design:  
16     - **Caller-driven snapshots** (no background thread): call ``snapshot(done, total)``  
from an  
17     existing progress callback; fps is derived from the delta since the previous snapshot.  
18     ``event(kind, msg)`` records discrete moments (start, video-done, error) WITH a health  
sample, so  
19     the last line before a kill captures the machine state at death.  
20     - **Capped + rotating**: a fixed ``meta`` header line (machine / GPU / start) is always
```

```

kept; recent
21     snapshot/event lines form an on-disk ring trimmed to ``max_bytes`` (~10 KB). The file
is rewritten
22     atomically (tmp + ``os.replace``) on each write, so a hard kill never leaves it half-
written.
23     - **Graceful degradation**: GPU stats come from ``nvidia-smi`` (absent -> ``None``);
system stats
24     from ``psutil`` (not installed -> ``None``). Collection never raises into the caller ?
a telemetry
25     hiccup must not break the run it observes.
26     - **Deterministic / testable**: the clock and the two collectors are injectable, so fps,
rotation,
27     and parsing are unit-tested with no real GPU, no psutil, and no wall-clock.
28
29     All stdlib + optional ``psutil`` (BSD-3, commercial-clean). No network, no training-data
path.
30     """
31     from __future__ import annotations
32
33     import json
34     import os
35     import shutil
36     import subprocess
37     import time
38     from typing import Any, Callable, Dict, List, Optional
39
40     GpuFn = Callable[[], Optional[Dict[str, Any]]]
41     SysFn = Callable[[], Optional[Dict[str, Any]]]
42
43     # nvidia-smi fields pulled in one query, in this order (see gpu_stats parsing).
44     _GPU_QUERY =
"temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free"
45     _THROTTLE_CLOCK_FRAC = 0.8 # SM clock below this fraction of max => flagged throttled
46     _THROTTLE_TEMP_C = 84.0 # ... or temperature at/above this (laptop Max-Q throttle
territory)
47
48
49     def _num(s: str) -> Optional[float]:
50         try:
51             return float(s)
52         except (TypeError, ValueError):
53             return None
54
55
56     def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
57         """One ``nvidia-smi`` sample as a dict, or ``None`` if nvidia-smi is unavailable /
errors.
58
59         Returns util %, temperature, power (W), current + max SM clock (MHz), VRAM used /
free (MB), and a
60         derived ``throttled`` flag (SM clock well below max, or hot). Pure read; never
raises.
61         """
62         exe = shutil.which("nvidia-smi")
63         if not exe:
64             return None
65         try:
66             out = subprocess.run(
67                 [exe, f"--query-gpu={_GPU_QUERY}", "--format=csv,noheader,nounits"],
68                 capture_output=True, text=True, timeout=timeout, check=True,
69             ).stdout.strip()
70         except (subprocess.SubprocessError, OSError):
71             return None
72         row = out.splitlines()[0] if out else ""
73         parts = [p.strip() for p in row.split(",")]

```

```

74     if len(parts) < 7:
75         return None
76     vals = [_num(p) for p in parts[:7]]
77     temp, util, power, sm, sm_max, used, free = vals
78     throttled = bool(temp is not None and temp >= _THROTTLE_TEMP_C)
79     if sm is not None and sm_max:
80         throttled = throttled or sm < _THROTTLE_CLOCK_FRAC * sm_max
81     return {
82         "util_pct": util, "temp_c": temp, "power_w": power,
83         "sm_mhz": sm, "sm_max_mhz": sm_max,
84         "vram_used_mb": used, "vram_free_mb": free,
85         "throttled": throttled,
86     }
87
88
89     def system_stats() -> Optional[Dict[str, Any]]:
90         """CPU % + RAM via ``psutil``, or ``None`` if psutil isn't installed. Never
91         raises."""
92         try:
93             import psutil # optional dep (BSD-3); only used when present
94         except ImportError:
95             return None
96         try:
97             vm = psutil.virtual_memory()
98             return {
99                 "cpu_pct": psutil.cpu_percent(interval=None),
100                 "ram_used_gb": round(vm.used / 1e9, 2),
101                 "ram_total_gb": round(vm.total / 1e9, 2),
102                 "ram_pct": vm.percent,
103             }
104         except Exception: # noqa: BLE001 - psutil edge cases must not break the run
105             return None
106
107     def gpu_name() -> Optional[str]:
108         """The GPU model string (for the runlog header), or ``None`` without nvidia-smi."""
109         exe = shutil.which("nvidia-smi")
110         if not exe:
111             return None
112         try:
113             out = subprocess.run([exe, "--query-gpu=name", "--format=csv,noheader"],
114                                 capture_output=True, text=True, timeout=4.0,
115                                 check=True).stdout
116         except (subprocess.SubprocessError, OSError):
117             return None
118         return out.splitlines()[0].strip() if out else None
119
120     class RunTelemetry:
121         """A small rotating JSONL runlog of execution velocity + machine health.
122
123         Call ``snapshot(done, total, phase=...)`` from a progress callback and ``event(kind,
124         msg)`` at
125         milestones. The file holds a ``meta`` header line plus the most recent snapshot /
126         event lines
127         that fit within ``max_bytes`` (~10 KB), rewritten atomically on every write.
128         """
129         def __init__(self, path: str, label: str = "run", max_bytes: int = 10_000,
130                     gpu_fn: GpuFn = gpu_stats, sys_fn: SysFn = system_stats,
131                     clock: Callable[[], float] = time.time) -> None:
132             self.path = path
133             self.label = label
134             self.max_bytes = max_bytes
135             self._gpu_fn = gpu_fn

```

```

135         self._sys_fn = sys_fn
136         self._clock = clock
137         self._t0 = clock()
138         self._last_done: Optional[int] = None
139         self._last_t: Optional[float] = None
140         self._lines: List[str] = [] # recent body lines (on-disk ring, trimmed to
max_bytes)
141         self._meta = json.dumps({"meta": {
142             "label": label, "started": self._iso(self._t0), "pid": os.getpid(),
143             "gpu_name": gpu_name(), "cpu_count": os.cpu_count(),
144         }}, separators=(",", ":"))
145         self._write()
146
147     # -- public API -----
148     def snapshot(self, done: Optional[int] = None, total: Optional[int] = None,
149                 phase: Optional[str] = None, **extra: Any) -> Dict[str, Any]:
150         """Record a velocity + health sample; ``fps`` is derived from the previous
snapshot."""
151         now = self._clock()
152         rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1)
153
154         if phase is not None:
155             rec["phase"] = phase
156         if done is not None:
157             rec["done"] = done
158         if total:
159             rec["total"] = total
160             rec["pct"] = round(100.0 * done / total, 1)
161         if self._last_done is not None and self._last_t is not None and now >
self._last_t:
162             rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
163             self._last_done, self._last_t = done, now
164         rec.update(extra)
165         self._attach_health(rec)
166         self._append(rec)
167         return rec
168
169     def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
170         """Record a milestone (start, video_done, error, restart) WITH a health
sample."""
171         now = self._clock()
172         rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1),
173
174             "event": kind}
175         if msg:
176             rec["msg"] = msg
177         rec.update(extra)
178         self._attach_health(rec)
179         self._append(rec)
180         return rec
181
182     # -- internals -----
183     def _attach_health(self, rec: Dict[str, Any]) -> None:
184         gpu = self._safe(self._gpu_fn)
185         sys_ = self._safe(self._sys_fn)
186         if gpu is not None:
187             rec["gpu"] = gpu
188         if sys_ is not None:
189             rec["sys"] = sys_
190
191     def _append(self, rec: Dict[str, Any]) -> None:
192         self._lines.append(json.dumps(rec, separators=(",", ":")))
193         budget = self.max_bytes - len(self._meta) - 1
194         while len(self._lines) > 1 and sum(len(s) + 1 for s in self._lines) > budget:
195             self._lines.pop(0) # drop oldest body line; the meta header is always kept

```

```

194         self._write()
195
196     def _write(self) -> None:
197         try:
198             tmp = self.path + ".tmp"
199             with open(tmp, "w", encoding="utf-8") as f:
200                 f.write(self._meta + "\n")
201                 for s in self._lines:
202                     f.write(s + "\n")
203             os.replace(tmp, self.path)
204         except OSError:
205             pass # telemetry must never break the run it observes
206
207     @staticmethod
208     def _safe(fn: GpuFn) -> Optional[Dict[str, Any]]:
209         try:
210             return fn()
211         except Exception: # noqa: BLE001 - a collector hiccup must not break the run
212             return None
213
214     @staticmethod
215     def _iso(t: float) -> str:
216         return time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(t))
217

```

4. user (2026-06-14T06:50:18.684Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7     CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8     measured by the shipped experiment harness; person features enter only as feature
columns the
9     learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse
20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates

```

```

31
32     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
33     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
34     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
35     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
36
37
38     def _derive_video_path(spec: Dict[str, Any]) -> str:
39         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
40         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
41         convention)."""
42         if spec.get("video_path"):
43             return str(spec["video_path"])
44         labels = str(spec["labels"])
45         if labels.endswith(".rallies.csv"):
46             return labels[: -len(".rallies.csv")] + ".MP4"
47         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
48         *.rallies.csv")
49
50     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                     frame_skip: int = 3, iou: float = 0.5,
52                     log_every_sec: float = 15.0) -> Dict[str, Any]:
53         """One golden video -> a feature record (shuttle + person features per candidate
54         window).
55         Returns `{name, fps, frame_width,
56         candidates:[{start,end,shuttle{,person{,label}], gts}``.
57         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
58         tests;
59         when None a real `PersonDetector` is built (loads torchvision on first use).
60         """
61         fps, fw = float(spec["fps"]), float(spec["frame_width"])
62         traj = str(spec["trajectory"])
63         points = TrackNetRunner.parse_trajectory_csv(traj)
64         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
65         gts = load_golden_gts(str(spec["labels"]))
66         labels = label_candidates(intervals, gts, iou)
67         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
68         estimate
69         # (parity with the shuttle net-crossing gate) rather than hard-coding an
70         orientation.
71         axis, net_px, _span = rally_gate.estimate_play_axis(points)
72         video_path = _derive_video_path(spec)
73
74         if detector is None:
75             from backend.pipeline.detectors.person_detector import PersonDetector
76             detector = PersonDetector({"indexing": {"use_gpu": True}})
77
78         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
79         # forward ? no per-window seek). Then compute features per window (pure, no GPU).
80         n = len(intervals)
81         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
82         print(f"[fusion_golden] {spec['name']}: {n} windows ? person-detecting in one
83         pass...", flush=True)
84         t0 = time.monotonic()
85         state = {"last": t0}
86
87         def _progress(done: int, total: int) -> None:
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0
91                 rate = done / el if el > 0 else 0.0
92                 eta = (total - done) / rate if rate > 0 else 0.0
93                 pct = 100 * done // total if total else 0

```

```

88         print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
89             f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
90         state["last"] = now
91
92     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
93                                                  progress_cb=_progress)
94
95     candidates: List[Dict[str, Any]] = []
96     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
97         det = per_window[i]
98         det_fw = det.get("frame_width") or fw
99         det_fh = det.get("frame_height") or 0.0
100        # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
101        # normalisation fusion_features uses (x/width, y/height).
102        if axis == "x":
103            net_norm = net_px / det_fw if det_fw > 0 else 0.5
104        else:
105            net_norm = net_px / det_fh if det_fh > 0 else 0.5
106        sf, ef = win_frames[i]
107        shuttle_in = [p for p in points if sf <= p.frame <= ef]
108        person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
109                                           court_polygon=None, net=(axis, net_norm))
110        candidates.append({
111            "start": float(s), "end": float(e),
112            "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
113            "person": {k: float(v) for k, v in person.items()},
114            "label": int(labels[i]),
115        })
116    return {
117        "name": spec["name"], "fps": fps, "frame_width": fw,
118        "candidates": candidates,
119        "gts": [[float(a), float(b)] for a, b in gts],
120    }
121
122
123    def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
124           fresh: bool = False, smallest_first: bool = False) -> List[str]:
125        with open(manifest_path, encoding="utf-8") as f:
126            specs = json.load(f)
127        if smallest_first:
128            # Process small videos first for quick feedback (the slow full match lands
last);
129            # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
130            specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
131                                                if os.path.isfile(s.get("trajectory", ""))
else 0))
132        os.makedirs(out_dir, exist_ok=True)
133        written: List[str] = []
134        for vi, spec in enumerate(specs, 1):
135            out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
136            # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
137            # run left off instead of re-detecting). --fresh forces a redo.
138            if not fresh and os.path.isfile(out):
139                print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
140                    f"-> {out} (use --fresh to redo)", flush=True)
141                written.append(out)
142                continue
143            print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
144            t0 = time.monotonic()

```

```

145         data = extract_video(spec, frame_skip=frame_skip, iou=iou)
146         n_pos = sum(c["label"] for c in data["candidates"])
147         with open(out, "w", encoding="utf-8") as f:
148             json.dump(data, f, indent=2)
149         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
150             f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
151             f"-> {out}", flush=True)
152         written.append(out)
153     return written
154
155
156     def main() -> None:
157         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
158         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
159         ap.add_argument("--out-dir", default="output")
160         ap.add_argument("--frame-skip", type=int, default=3)
161         ap.add_argument("--iou", type=float, default=0.5)
162         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
163         ap.add_argument("--smallest-first", action="store_true",
164             help="process smaller videos first for quick feedback (LOVO is
order-independent)")
165         args = ap.parse_args()
166         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
167             smallest_first=args.smallest_first)
168
169
170     if __name__ == "__main__":
171         main()
172

```

5. user (2026-06-14T06:50:19.119Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.
28    - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29      variants agree / diverge (A-only, B-only, agreed). The divergences are what a

```



```

30     human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32     (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Callable, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54 from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55 from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
ratio
56 from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
scores
57
58 logger = logging.getLogger(__name__)
59
60 # Where a comparison run appends one reproducible record. Tracked location (NOT under
61 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62 # regression.DEFAULT_BASELINE_PATH.
63 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64 _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67 class ExperimentError(ValueError):
68     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69     score an unlabeled video with no pinned model, or a gate referencing an absent
70     feature)."""
71
72
73 # ----- Variant
74
75
76 @dataclass
77 class Variant:
78     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80     A variant turns one video's persisted candidate windows + features into rally
81     intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82     filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83     to control ? the core no-regression guarantee.
84
85     Fields:
86     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87       for the leaderboard and for selecting the served path (the existing
88       `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89       New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate **Gate A** : keep only windows whose persisted
91       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92       `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-

```

```

shuttle
93         windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98       `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for `compare_unlabeled` on a learned variant.
100    - ``feature_names`` ? the persisted features the learned filter consumes. When set
101      WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102      (honest) ? the recipe, not a pinned artifact.
103    - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104      overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105    ""
106
107    name: str
108    segmenter: str = "wasb_hybrid"
109    config_overrides: Dict[str, Any] = field(default_factory=dict)
110    cue_flags: Dict[str, bool] = field(default_factory=dict)
111    min_crossings: int = 0
112    min_players: int = 0
113    classifier_model: Optional[str] = None
114    feature_names: Optional[List[str]] = None
115    threshold: float = 0.5
116    merge_gap: float = 1.0
117    # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118    # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119    # failure where a fixed 0.5 zeroed out a small-candidate video.
120    tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121    calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123    def is_learned(self) -> bool:
124        """True if this variant applies a learned classifier (pinned model or
recipe)."""
125        return self.classifier_model is not None or bool(self.feature_names)
126
127    def to_dict(self) -> dict:
128        return {
129            "name": self.name,
130            "segmenter": self.segmenter,
131            "config_overrides": self.config_overrides,
132            "cue_flags": self.cue_flags,
133            "min_crossings": self.min_crossings,
134            "min_players": self.min_players,
135            "classifier_model": self.classifier_model,
136            "feature_names": self.feature_names,
137            "threshold": self.threshold,
138            "merge_gap": self.merge_gap,
139            "tune_threshold": self.tune_threshold,
140            "calibrate": self.calibrate,
141        }
142
143    @classmethod
144    def from_dict(cls, d: dict) -> "Variant":
145        known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146        return cls(**{k: v for k, v in d.items() if k in known})
147
148    def spec_hash(self) -> str:
149        """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150        and makes a result reproducible. The pinned **control** variant always hashes
the
151        same, so a regression is always traceable to a recipe change, never to drift."""

```

```

152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)
204     mask = [True] * n
205     if variant.min_crossings > 0:
206         col = _feature_column(vc, "net_crossings")
207         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208     if variant.min_players > 0:
209         col = _feature_column(vc, "players_in_court")
210         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211     return mask
212
213
214 def predict(variant: Variant, vc: VideoCandidates,
215             model: Optional[LogisticRegression] = None,

```

```

215         threshold: Optional[float] = None,
216         calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217     """Variant prediction: gate by persisted features, optionally filter by a learned
218     model, then merge. Pure and deterministic.
219
220     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225     them.
226     """
227     mask = _gate_mask(variant, vc)
228     if variant.feature_names:
229         if model is None:
230             if variant.classifier_model is None:
231                 raise ExperimentError(
232                     f"variant {variant.name!r} is learned but has no model to predict "
233                     f"with (pass a fitted model or set classifier_model)")
234             model = LogisticRegression.load(variant.classifier_model)
235             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
236 variant.feature_names))]
237             if calibrator is not None:
238                 proba = [calibrator(p) for p in proba]
239                 thr = variant.threshold if threshold is None else threshold
240                 mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
241             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
242             return merge_overlaps(kept, gap_tol=variant.merge_gap)
243
244     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
245                             train_videos: Sequence[VideoCandidates], iou: float
246                             ) -> "tuple[Optional[Callable[[float], float]],
247 Optional[float]]":
248         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
249         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
250         (use the variant default). Honest: never looks at the held-out video.
251         """
252         calibrator: Optional[Callable[[float], float]] = None
253         if variant.calibrate:
254             raw: List[float] = []
255             y: List[int] = []
256             for vc in train_videos:
257                 raw.extend(float(p) for p in
258                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
259 [])))
260                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
261             if variant.calibrate == "platt":
262                 calibrator = fit_platt(raw, y)
263             elif variant.calibrate == "isotonic":
264                 calibrator = fit_isotonic(raw, y)
265             else:
266                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
267 'platt'/'isotonic')")
268             threshold: Optional[float] = None
269             if variant.tune_threshold:
270                 best_t, best_f1 = variant.threshold, -1.0
271                 for k in range(1, 20):
272                     # grid 0.05..0.95 on the train
273                     t = round(0.05 * k, 2)
274                     f1s = [score(predict(variant, vc, model=model, threshold=t,
275 calibrator=calibrator),
276                               vc.gts or [], iou).f1 for vc in train_videos]
277                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
278                     if mean_f1 > best_f1:

```

```

273         best_f1, best_t = mean_f1, t
274         threshold = best_t
275     return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (`classifier_model` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:

```

```

331         raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332         model: Optional[LogisticRegression] = _train(variant, others, iou)
333         if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334             assert model is not None # _train never returns
None
335             cal, thr = _calibrate_and_tune(variant, model, others, iou)
336         elif recipe_lovo: # honest=False ? one model over all
videos
337             model = all_model
338         else: # static variant or pinned model
339             model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
for m in _METRICS}
347         for m in _METRICS}
348         return {
349             "variant": variant.name,
350             "spec_hash": variant.spec_hash(),
351             "is_learned": variant.is_learned(),
352             "honest_lovo": recipe_lovo and honest,
353             "per_video": per_video,
354             "aggregate": aggregate,
355             "predictions": predictions,
356         }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],

```

```

385         eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                             "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
```

```

435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451 # ----- SxS on unlabeled video
452
453
454 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                      agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471
472     """
473     preds_a = predict(variant_a, video)
474     preds_b = predict(variant_b, video)
475     mr = match_intervals(preds_a, preds_b, agree_iou)
476
477     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
478              for m in mr.matches]
479     agree_ious = [m.iou for m in mr.matches]
480     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
481     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
482
483     def _dur(ivs: List[Interval]) -> float:
484         return sum(e - s for s, e in ivs)
485
486     return {
487         "video": video.name,
488         "agree_iou": agree_iou,
489         "variant_a": variant_a.name,
490         "variant_b": variant_b.name,
491         "num_a": len(preds_a),
492         "num_b": len(preds_b),
493         "num_agreed": len(agreed),
494         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
495         "duration_a": _dur(preds_a),
496         "duration_b": _dur(preds_b),
497         "agreed": agreed,
498         "a_only": a_only,
499         "b_only": b_only,
500     }

```



```

499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     # the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)
535     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536     with open(path, "w", encoding="utf-8") as f:
537         json.dump(entries, f, indent=2)
538     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
539               row["variant_a"]["name"], row["variant_b"]["name"],
540               (result.get("delta") or {}).get("f1", 0.0), path)
541     return row
542
543 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                          gts: Optional[List[Interval]] = None,
545                          feature_names: Sequence[str] = ()) -> VideoCandidates:
546     """Bridge persisted detection ? `VideoCandidates` via
547     `Database.query_candidate_segments`.
548     ``feature_names`` selects which persisted features to lift out of each row's
549     ``stats``
550     JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551     touching
552     function; the comparison itself is pure and offline. Exercised owner-side on a real
553     DB,
554     but works against any `Database` (including an in-memory one in tests).
555     """
556     rows = db.query_candidate_segments(video_id, detector=detector)
557     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558                                rows]
559     features: Dict[str, List[float]] = {}
560     for name in feature_names:
561         if name == "duration":

```

```

558         features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559     else:
560         features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561     return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

6. assistant (2026-06-14T06:50:22.085Z)

Now let me check the CI gates to complete the picture.

7. user (2026-06-14T06:50:22.611Z)

```
.github\workflows\ci.yml
```

8. user (2026-06-14T06:50:22.765Z)

No files found

9. user (2026-06-14T06:50:31.116Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11     concurrency:
12         group: ci-${{ github.ref }}
13         cancel-in-progress: true
14
15     jobs:
16         # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17         # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18         # tracked .py, including scripts/ and training/, which the backend-only
19         # py_compile smoke sweep does not reach.
20         lint:
21             name: Ruff lint
22             runs-on: ubuntu-latest
23             steps:
24                 - uses: actions/checkout@v6
25                 - uses: actions/setup-python@v6
26                   with:
27                       python-version: "3.13"
28                 - name: Install ruff
29                   run: python -m pip install ruff
30                 - name: Ruff check
31                   run: ruff check . --output-format github
32
33         # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34         # per-module quarantine list ? ratchet by removing entries as modules clean
35         # up. Typed core deps are installed so mypy sees their real signatures
36         # (without them ignore_missing_imports would Any-stub half the checks).
37         typecheck:
38             name: Mypy (lenient baseline)
39             runs-on: ubuntu-latest
40             steps:
41                 - uses: actions/checkout@v6

```

```

42         - uses: actions/setup-python@v6
43         with:
44             python-version: "3.13"
45         - name: Install mypy + typed core deps
46         run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47         - name: Mypy
48         run: mypy backend training
49
50     # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
51     # module-level import error shipping while the suite stays green.
52     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53     # on machines without the GPU stack (test_import_smoke stubs what's absent).
54     import-smoke:
55         name: Syntax sweep + import smoke (no GPU stack)
56         runs-on: ubuntu-latest
57         steps:
58             - uses: actions/checkout@v6
59             - uses: actions/setup-python@v6
60             with:
61                 python-version: "3.13"
62             - name: Install minimal deps
63             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64             - name: Run import smoke tests
65             run: python -m pytest tests/test_import_smoke.py -v
66
67     full-suite:
68         name: Full test suite (offline, mocked)
69         runs-on: ubuntu-latest
70         steps:
71             - uses: actions/checkout@v6
72             - uses: actions/setup-python@v6
73             with:
74                 python-version: "3.13"
75                 cache: pip
76             - name: System libs for opencv-python
77             run: |
78                 sudo apt-get update
79                 sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
80             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81             run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
82             - name: Install requirements
83             run: python -m pip install -r requirements.txt pytest pytest-cov httpx # httpx:
fastapi TestClient
84             # obsreport (private, soft dep) is absent here: reporting tests skip via
85             # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
86             # is therefore calibrated against THIS lane's number (local runs with
87             # obsreport installed measure a little higher).
88             # Floor calibrated 2026-06-11 against this lane's measured 72% (local
89             # with obsreport: 73%). Ratchet it UP as coverage grows; never down
90             # without an owner decision.
91             - name: Run full suite (with coverage floor)
92             run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
93

```

10. assistant (2026-06-14T06:50:36.013Z)

I have everything I need. Let me compose the structured answer.

11. user (2026-06-14T06:51:20.501Z)

Structured output provided successfully

12. assistant (2026-06-14T06:51:33.107Z)

I've extracted the engineering + reproducibility story and returned it via StructuredOutput. The four pillars:

1. ****Replayable feature substrate**** (`fusion\_golden.py`): GPU runs once per video; person detection is the only GPU touch. Reuses existing trajectory CSVs (no WASB re-run), no Gemini/DB. Each `\_golden\_features.json` lets `fusion\_compare` re-score with any variant/threshold forever ? "persist once, re-score forever."
2. ****Experiment leaderboard**** (`experiment.py`): the detection/decision split makes the DECISION layer a pure, deterministic, declarative `Variant` recipe. `record\_experiment` appends pinned rows (variant `spec\_hash` + `label\_set\_hash` + IoU) to a tracked `eval\_baselines/experiment\_leaderboard.json` that survives a clean checkout. LOVO + bootstrap CIs + paired sign test keep the lift honest.
3. ****~10KB rotating telemetry black box**** (`run\_telemetry.py`): atomic-write ring of fps + GPU temp/clocks/VRAM/throttle + CPU/RAM, last line captures machine state at a traceback-less kill. Never raises into the run; collectors/clock injectable for tests.
4. ****Default-OFF + CI-gated discipline****: every Variant knob defaults to control (a carried-but-disabled cue is byte-identical to control); CI runs ruff lint, mypy lenient baseline, no-GPU syntax+import smoke, and the full offline-mocked suite at a 70% coverage floor.

All file paths and the load-bearing details are in the structured output.